Compile the file *testlib.c* to generate a shared library, with the following command:

```
gcc -shared -fPIC -o testlib.so testlib.c
```

You should find the new shared library *testlib.so* in the current working directory. To make a call to the C function in the library from Python, run the Python file *testlib.py*.

Note: Make sure that the dynamic runtime linker finds the shared libraries created in the various code examples shown in the article—otherwise the programs will throw errors. Open a terminal and navigate (*cd*) to the downloaded source code folder to make it your current working directory. Now run the following command:

```
export LD_LIBRARY_PATH=.:$LD_LIBRARY_PATH
```

The above command will add your current working directory to the *LD_LIBRARY_PATH*. However, note that this is a temporary solution. To make the change permanent, edit your *~.bashrc* file with the relavent information.

Note how we passed the integer data from Python to the C function: for each integer, we used *c_int()* to create the integer object, and passed a pointer to it using *byref()*. (That is because the shared library expects a pointer to an integer, because it modifies the value stored in it.) We use the *value* attribute of the integer object to retrieve the value set by the *dataModel()* function in *testlib.c*. You can follow the same method for any supported data object/type.

You can also manipulate C arrays, structures, unions, pointers, etc, through Ctypes. To explore more of these, follow the Ctypes link in the *References* section.

SWIG—a wrapper generator for Python extension modules

Ctypes is limited in some ways: you can extend Python only with functions written in C. Also, you need to create Python 'wrapper' code around the call to the function, with the necessary conversions between Python types and Ctypes data objects.

SWIG, the Simplified Wrapper and Interface Generator, is another way to extend Python through shared libraries. It's a powerful tool that can automatically generate Python bindings for C and C++ sources. Google extensively uses SWIG, which is proof of its capabilities. SWIG is not limited to Python use only: it can wrap C and C++ functionality for use in more than a dozen programming languages, including Ruby, Perl, PHP, Lua, Tcl and Java.

The easiest way to install SWIG on Ubuntu is to run the following command:

```
sudo apt-get install swig
```

To install SWIG from source, you need GCC and g++ (install the *build-essential* metapackage). Download the SWIG source tarball from its homepage (see *References*) and then run these commands:

```
tar -zxvf swig-version.tar.gz && cd swig-version
./configure && make && sudo make install
```

You also need Python development headers installed to create extension modules with SWIG. Now run the following:

```
sudo apt-get install python-dev
```

SWIG follows a layered approach to generate Python extension modules from C/C++ sources: it generates a C file that contains the lower-level code required for the extension module, and a Python file that contains the higher-level code. To generate an extension module from your C/C++ source, first you need to prepare an *interface file* that provides SWIG with information about the declaration and definitions of your data structures and routines. SWIG generates layered wrapper files from the interface file. Then you turn the generated C wrapper file and your source file into a shared library, and use your extension library through the generated Python wrapper (see the example session below). You could also automate generation of the shared library using Python's inbuilt *distutils* package.

Let's see SWIG in action, with a trivial C++ extension module. In the working directory where you extracted the source files archive, take a look at *testmodule.i*, *testmodule.hpp* and *test.py*.

The SWIG interface file *testmodule.i* has the directive *%module* that specifies the extension module you intend to generate. You could also provide the module name with the *-module* switch to SWIG if you don't provide this directive in the interface file. The text between *%{* and *%}* is put in the generated C/C++ wrapper file verbatim, so it is the place for all macros, headers, etc, required to build the module library. Declaration of module data structures and routines is done below this section. SWIG generates the wrapper files based upon these signatures.

Run the command *swig -python -c++ testmodule.i* to generate the wrapper files *testmodule_wrap.cxx* and *testmodule.py* in the working directory. Run *g++ -shared -fPIC -o testmodule.so testmodule.cpp testmodule_wrap.cxx -I(location of Python headers)* to build the shared extension library. Finally, execute *test.py* to access the C++ routine *helloSwig()* from Python.

The *-c++* and *-python* switches instruct SWIG to generate a C++ wrapper for Python. You can change the wrapper filename from the default *modulename_wrap* with the *-o* switch. Also note that the name of the generated shared library is *_modulename.so*, as this is the Python naming convention for extension modules, and the generated Python wrapper module looks for it under this name.

Now we try something more serious. Look at the files *testclass.i*, *testclass.hpp*, *testclass.cpp* and *testoop.py*. Generate the extension shared library as described for the previous example. Run *testoop.py* to instantiate and access a simple C++ class from Python code.